

Vietnam National University, Ho Chi Minh City
University of Technology
Faculty of Computer Science and Engineering



Microprocessors-Microcontrollers (Lab) (CO3010)

Assignment

LAB_01

Instructor(s): Phan Văn Sỹ, CSE-HCMUT
Class: CC03
Student: Võ Lê Thành Triệu - 2453297

Ho Chi Minh City, September 2026



Contents

1	LED Animation Exercises	2
1.1	Exercise 1: Alternating Two LEDs	2
1.1.1	Objective and GPIO configuration	2
1.1.2	Schematic	2
1.1.3	Implementation and operation	3
1.2	Exercise 2: Three-LED Traffic Light	4
1.2.1	Objective and GPIO configuration	4
1.2.2	Schematic	4
1.2.3	Implementation and operation	5
1.3	Exercise 3: Four-Way Traffic Light	5
1.3.1	Objective and control strategy	5
1.3.2	Schematic	6
1.3.3	Traffic-light sequence	7
1.3.4	Implementation and operation	7
1.4	Exercise 4: Seven-Segment Display	8
1.4.1	Objective and GPIO configuration	8
1.4.2	Schematic	9
1.4.3	Implementation and operation	9
1.5	Exercise 5: Four-Way Traffic Light with Countdown	12
1.5.1	Objective and system integration	12
1.5.2	Schematic	13
1.5.3	Implementation and operation	14
1.6	Exercise 6: Analog Clock LED Connection Test	16
1.6.1	Objective and GPIO configuration	16
1.6.2	Schematic	17
1.6.3	Implementation and operation	18
1.7	Exercise 7: Clearing All Clock LEDs	19
1.7.1	Objective	19
1.7.2	Implementation and operation	19
1.8	Exercise 8: Activating a Selected Clock LED	20
1.8.1	Objective	20
1.8.2	Implementation and operation	21
1.9	Exercise 9: Deactivating a Selected Clock LED	23
1.9.1	Objective	23
1.9.2	Implementation and operation	24
1.10	Exercise 10: Integrated Analog Clock	26
1.10.1	Objective and control strategy	26
1.10.2	Implementation and operation	27

1 LED Animation Exercises

This report presents the design and implementation of ten LED animation exercises using the STM32F103C6Ux microcontroller [3]. STM32CubeMX was used to configure the GPIO pins and generate the initialization code [4]. The application code was edited and compiled in Visual Studio Code using CMake, while Proteus was used to simulate the circuits and firmware [6].

Throughout these exercises, the discrete LEDs are connected using active-low logic: the positive terminal is connected to the 3.3+V supply and the negative terminal is connected to an STM32 GPIO output. Therefore, a low output activates an LED and a high output deactivates it. The common-anode seven-segment display used in Exercises 4 and 5 follows the same active-low control principle.

The submitted Proteus schematics use direct simulated connections and do not contain external series resistors. This description refers specifically to the Proteus models used in this laboratory.

Table 1: Relationship between the GPIO output and an active-low LED

HAL state	Approximate output voltage	LED state
GPIO_PIN_RESET	0 V	On
GPIO_PIN_SET	3.3 V	Off

1.1 Exercise 1: Alternating Two LEDs

1.1.1 Objective and GPIO configuration

Exercise 1 controls a red LED connected to PA5 and a yellow LED connected to PA6. The two LEDs must alternate every two seconds. When the red LED is on, the yellow LED is off, and after two seconds their states are exchanged.

Table 2: GPIO configuration for Exercise 1

GPIO pin	CubeMX label	Proteus component	Function
PA5	LED_RED	D1	Red LED output
PA6	LED_YELLOW	D2	Yellow LED output

1.1.2 Schematic

Figure 1 presents the Proteus schematic of Exercise 1. All source code and Proteus project files are stored in the common Lab 01 repository listed in Reference [1].

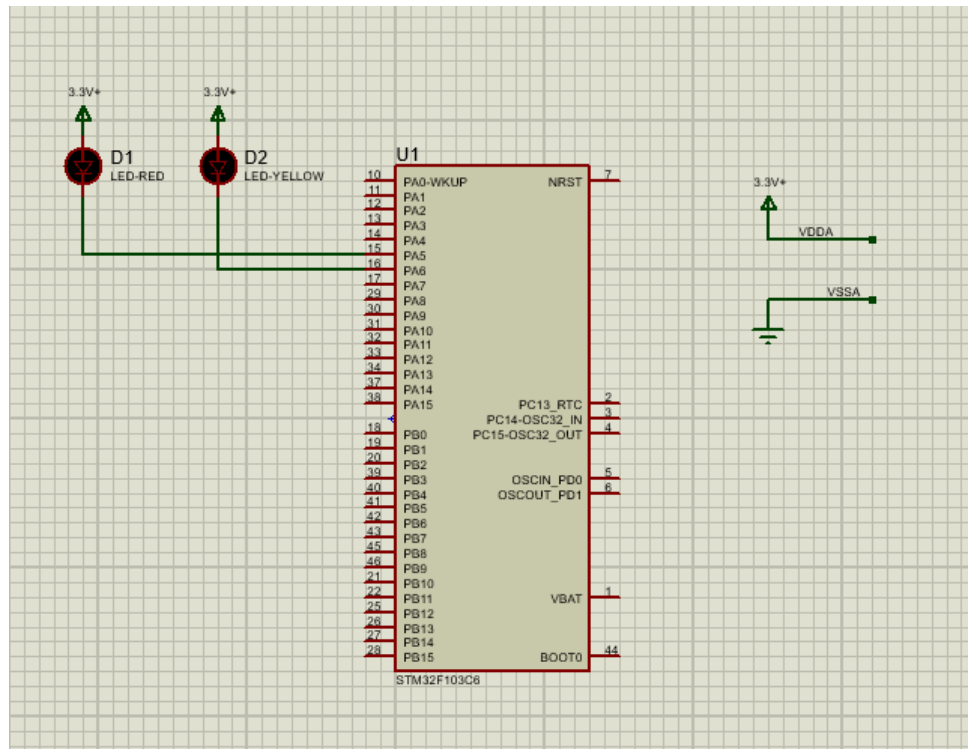


Figure 1: Exercise 1 Proteus schematic. Project files are provided in Reference [1].

1.1.3 Implementation and operation

The program first writes a low level to PA5 to activate the red LED and a high level to PA6 to deactivate the yellow LED. After a 2000-ms delay, the output levels are reversed. Because these instructions are placed inside an infinite loop, the two states repeat continuously.

Program 1.1: Alternating red and yellow LEDs

```
1 while (1)
2 {
3     /* State 1: Red ON, Yellow OFF */
4     HAL_GPIO_WritePin(LED_RED_GPIO_Port,
5                         LED_RED_Pin,
6                         GPIO_PIN_RESET);
7
8     HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port,
9                         LED_YELLOW_Pin,
10                        GPIO_PIN_SET);
11
12     HAL_Delay(2000);
13
14     /* State 2: Red OFF, Yellow ON */
15     HAL_GPIO_WritePin(LED_RED_GPIO_Port,
16                         LED_RED_Pin,
17                         GPIO_PIN_SET);
18
19     HAL_GPIO_WritePin(LED_YELLOW_GPIO_Port,
20                         LED_YELLOW_Pin,
21                         GPIO_PIN_RESET);
```

```

22
23     HAL_Delay(2000);
24 }

```

The resulting sequence is summarized in Table 3.

Table 3: State sequence for Exercise 1

State	Red LED	Yellow LED	Duration
1	On	Off	2 seconds
2	Off	On	2 seconds

1.2 Exercise 2: Three-LED Traffic Light

1.2.1 Objective and GPIO configuration

Exercise 2 extends the previous circuit by adding a green LED to PA7. The three LEDs simulate a simple traffic light. The red state lasts five seconds, the green state lasts three seconds, and the yellow state lasts two seconds.

Table 4: GPIO configuration for Exercise 2

GPIO pin	CubeMX label	Proteus component	Function
PA5	LED_RED	D1	Red LED output
PA6	LED_YELLOW	D2	Yellow LED output
PA7	LED_GREEN	D3	Green LED output

1.2.2 Schematic

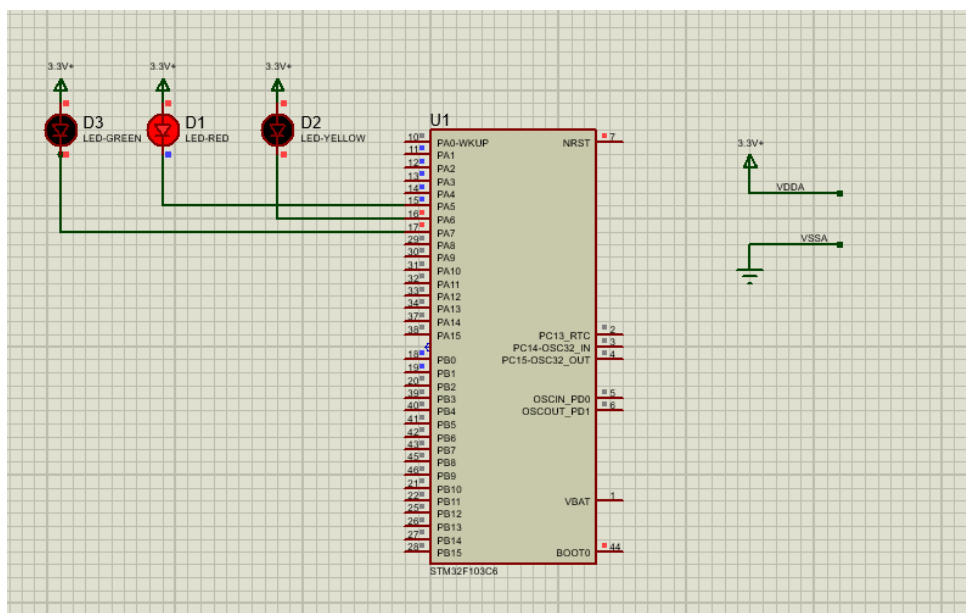


Figure 2: Exercise 2 Proteus schematic. Project files are provided in Reference [1].

1.2.3 Implementation and operation

Only one LED is activated in each traffic-light state. Each state is written explicitly so that the required output levels can be inspected easily. This approach also guarantees that the two unused LEDs are switched off before the program waits for the specified duration.

Program 1.2: Three-state traffic-light controller

```

1  while (1)
2  {
3      /* RED for 5 seconds */
4      HAL_GPIO_WritePin(GPIOA, LED_RED_Pin,
5                          GPIO_PIN_RESET);
6      HAL_GPIO_WritePin(GPIOA, LED_YELLOW_Pin,
7                          GPIO_PIN_SET);
8      HAL_GPIO_WritePin(GPIOA, LED_GREEN_Pin,
9                          GPIO_PIN_SET);
10
11     HAL_Delay(5000);
12
13     /* GREEN for 3 seconds */
14     HAL_GPIO_WritePin(GPIOA, LED_RED_Pin,
15                         GPIO_PIN_SET);
16     HAL_GPIO_WritePin(GPIOA, LED_YELLOW_Pin,
17                         GPIO_PIN_SET);
18     HAL_GPIO_WritePin(GPIOA, LED_GREEN_Pin,
19                         GPIO_PIN_RESET);
20
21     HAL_Delay(3000);
22
23     /* YELLOW for 2 seconds */
24     HAL_GPIO_WritePin(GPIOA, LED_RED_Pin,
25                         GPIO_PIN_SET);
26     HAL_GPIO_WritePin(GPIOA, LED_YELLOW_Pin,
27                         GPIO_PIN_RESET);
28     HAL_GPIO_WritePin(GPIOA, LED_GREEN_Pin,
29                         GPIO_PIN_SET);
30
31     HAL_Delay(2000);
32 }

```

One complete traffic-light cycle lasts ten seconds, as shown in Table 5.

Table 5: State sequence for Exercise 2

State	Red	Yellow	Green	Duration
1	On	Off	Off	5 seconds
2	Off	Off	On	3 seconds
3	Off	On	Off	2 seconds

1.3 Exercise 3: Four-Way Traffic Light

1.3.1 Objective and control strategy

Exercise 3 expands the circuit into a four-way traffic light containing twelve physical LEDs. Each direction has one red, one yellow, and one green LED. In the implemented two-phase in-

tersection, opposite directions are synchronized: North and South always display the same state, while East and West always display the same state.

This design uses six GPIO control signals. Each output controls two physical LEDs of the same color in opposite directions. Therefore, the system still contains all twelve required LEDs:

$$6 \text{ GPIO control signals} \times 2 \text{ physical LEDs per signal} = 12 \text{ physical LEDs.}$$

The North-South and East-West groups must never display green simultaneously because their traffic paths intersect. While one group is green or yellow, the perpendicular group remains red.

Table 6: GPIO configuration for Exercise 3

GPIO pin	CubeMX label	Proteus components	Physical LEDs controlled
PA5	NS_RED	D1, D4	North red and South red
PA6	NS_YELLOW	D2, D5	North yellow and South yellow
PA7	NS_GREEN	D3, D6	North green and South green
PA8	EW_RED	D7, D10	East red and West red
PA9	EW_YELLOW	D8, D11	East yellow and West yellow
PA10	EW_GREEN	D9, D12	East green and West green

In the submitted Proteus schematic, each LED is connected directly to its assigned control signal and no external resistor component is used.

1.3.2 Schematic

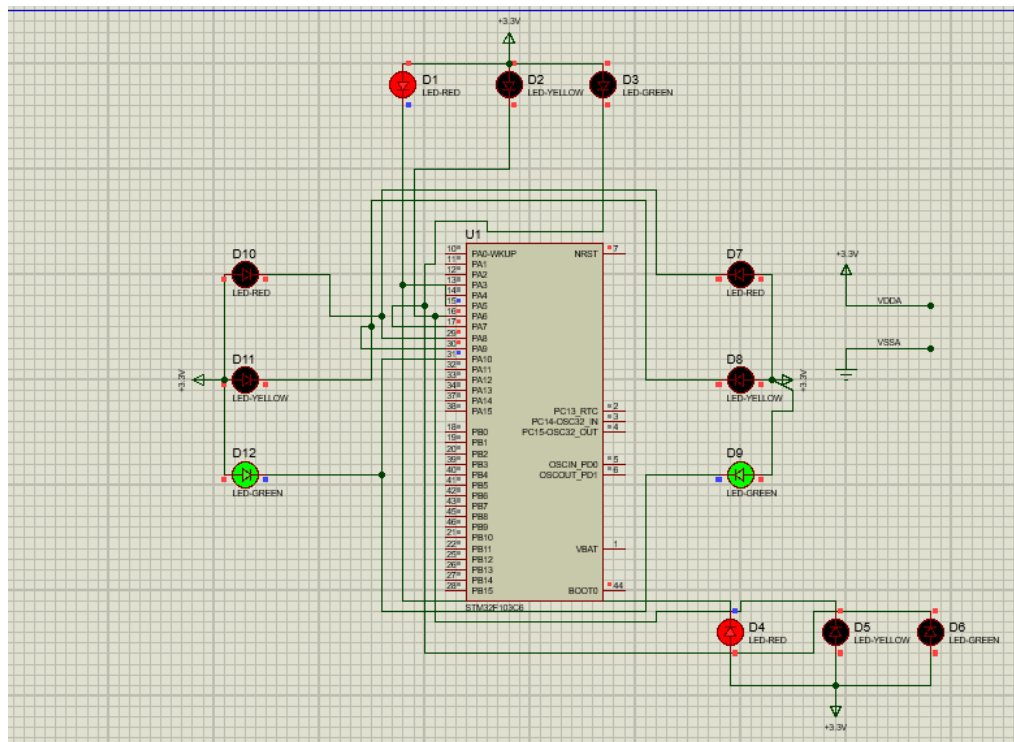


Figure 3: Exercise 3 Proteus schematic. Project files are provided in Reference [1].

1.3.3 Traffic-light sequence

The four-way intersection uses four states. Each direction receives a three-second green state and a two-second yellow state. Its red state lasts five seconds, which is equal to the other group's green and yellow durations.

Table 7: State sequence for the four-way traffic light

State	North-South	East-West	Duration
1	Green	Red	3 seconds
2	Yellow	Red	2 seconds
3	Red	Green	3 seconds
4	Red	Yellow	2 seconds

1.3.4 Implementation and operation

The program writes every output explicitly to keep the implementation easy to read and debug. A selected color is activated with GPIO_PIN_RESET; the other colors are deactivated with GPIO_PIN_SET.

Program 1.3: Four-way traffic-light controller

```

1  while (1)
2  {
3      /* State 1: North-South GREEN, East-West RED */
4      HAL_GPIO_WritePin(GPIOA, NS_RED_Pin,    GPIO_PIN_SET);
5      HAL_GPIO_WritePin(GPIOA, NS_YELLOW_Pin,  GPIO_PIN_SET);
6      HAL_GPIO_WritePin(GPIOA, NS_GREEN_Pin,   GPIO_PIN_RESET);
7
8      HAL_GPIO_WritePin(GPIOA, EW_RED_Pin,     GPIO_PIN_RESET);
9      HAL_GPIO_WritePin(GPIOA, EW_YELLOW_Pin,  GPIO_PIN_SET);
10     HAL_GPIO_WritePin(GPIOA, EW_GREEN_Pin,   GPIO_PIN_SET);
11
12     HAL_Delay(3000);
13
14     /* State 2: North-South YELLOW, East-West RED */
15     HAL_GPIO_WritePin(GPIOA, NS_RED_Pin,    GPIO_PIN_SET);
16     HAL_GPIO_WritePin(GPIOA, NS_YELLOW_Pin,  GPIO_PIN_RESET);
17     HAL_GPIO_WritePin(GPIOA, NS_GREEN_Pin,   GPIO_PIN_SET);
18
19     HAL_GPIO_WritePin(GPIOA, EW_RED_Pin,     GPIO_PIN_RESET);
20     HAL_GPIO_WritePin(GPIOA, EW_YELLOW_Pin,  GPIO_PIN_SET);
21     HAL_GPIO_WritePin(GPIOA, EW_GREEN_Pin,   GPIO_PIN_SET);
22
23     HAL_Delay(2000);
24
25     /* State 3: North-South RED, East-West GREEN */
26     HAL_GPIO_WritePin(GPIOA, NS_RED_Pin,    GPIO_PIN_RESET);
27     HAL_GPIO_WritePin(GPIOA, NS_YELLOW_Pin,  GPIO_PIN_SET);
28     HAL_GPIO_WritePin(GPIOA, NS_GREEN_Pin,   GPIO_PIN_SET);
29
30     HAL_GPIO_WritePin(GPIOA, EW_RED_Pin,     GPIO_PIN_SET);
31     HAL_GPIO_WritePin(GPIOA, EW_YELLOW_Pin,  GPIO_PIN_SET);
32     HAL_GPIO_WritePin(GPIOA, EW_GREEN_Pin,   GPIO_PIN_RESET);
33
34     HAL_Delay(3000);

```

```

35
36      /* State 4: North-South RED, East-West YELLOW */
37      HAL_GPIO_WritePin(GPIOA, NS_RED_Pin,    GPIO_PIN_RESET);
38      HAL_GPIO_WritePin(GPIOA, NS_YELLOW_Pin,  GPIO_PIN_SET);
39      HAL_GPIO_WritePin(GPIOA, NS_GREEN_Pin,   GPIO_PIN_SET);
40
41      HAL_GPIO_WritePin(GPIOA, EW_RED_Pin,     GPIO_PIN_SET);
42      HAL_GPIO_WritePin(GPIOA, EW_YELLOW_Pin,  GPIO_PIN_RESET);
43      HAL_GPIO_WritePin(GPIOA, EW_GREEN_Pin,   GPIO_PIN_SET);
44
45      HAL_Delay(2000);
46  }

```

After the fourth state, the infinite loop returns to the first state. This creates a continuous ten-second intersection cycle without allowing the two perpendicular traffic groups to receive green signals at the same time.

1.4 Exercise 4: Seven-Segment Display

1.4.1 Objective and GPIO configuration

Exercise 4 adds a common-anode seven-segment display to the circuit developed in Exercise 3. The seven segments are connected to PB0 through PB6. The program displays the decimal numbers from 0 to 9, with each number remaining visible for one second.

Because the display is common-anode, its segments are active-low. A segment is activated using `GPIO_PIN_RESET` and deactivated using `GPIO_PIN_SET`.

Table 8: GPIO configuration for Exercise 4

GPIO pin	CubeMX label	7SEG pin	Function
PB0	SEG_A	a	Segment a output
PB1	SEG_B	b	Segment b output
PB2	SEG_C	c	Segment c output
PB3	SEG_D	d	Segment d output
PB4	SEG_E	e	Segment e output
PB5	SEG_F	f	Segment f output
PB6	SEG_G	g	Segment g output

PB3 and PB4 are normally reserved for the JTAG debugging interface. Therefore, the debug mode in STM32CubeMX is configured as **Serial Wire**. This releases PB3 and PB4 for GPIO operation while keeping the SWD programming and debugging interface available.

1.4.2 Schematic

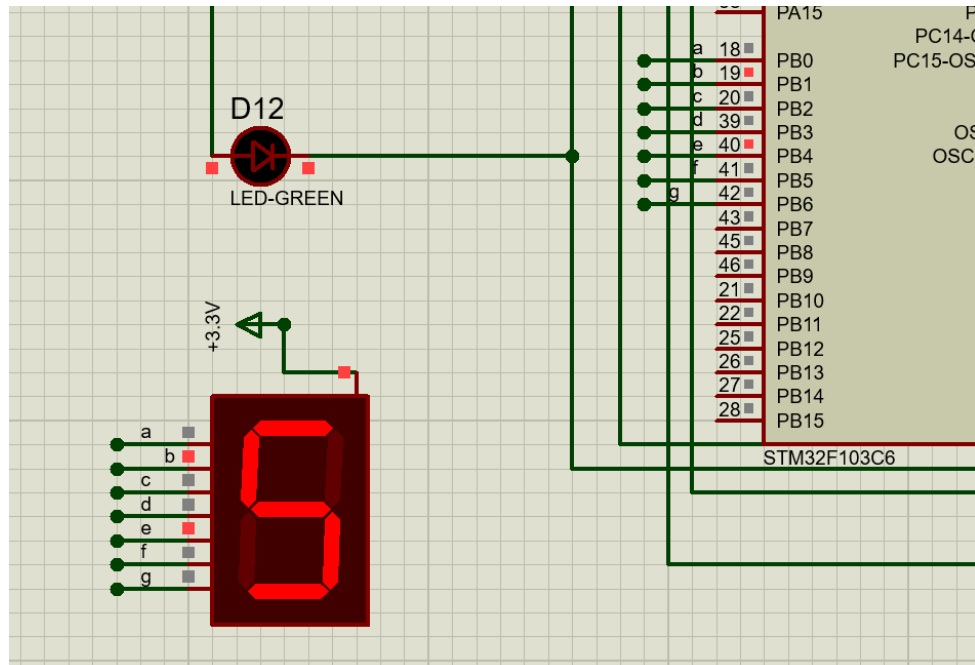


Figure 4: Exercise 4 Proteus schematic (if you want to see the full schematic check out for exercise 05 figure). Project files are provided in Reference [1].

1.4.3 Implementation and operation

The function `display7SEG()` receives an integer named `num`. It first deactivates all seven segments to remove the previously displayed number. A `switch` statement then selects and activates the segments required for the new number.

All segment pins belong to GPIO port B. The bitwise OR operator (`|`) combines several GPIO pin masks, allowing multiple segments to be controlled by one call to `HAL_GPIO_WritePin()`.

Program 1.4: Seven-segment display function

```
1 void display7SEG(int num)
2 {
3     /*
4      * Turn every segment OFF first.
5      * Common-anode display: SET = OFF.
6      */
7     HAL_GPIO_WritePin(
8         GPIOB,
9         SEG_A_Pin | SEG_B_Pin | SEG_C_Pin |
10        SEG_D_Pin | SEG_E_Pin | SEG_F_Pin |
11        SEG_G_Pin,
12        GPIO_PIN_SET
13    );
14
15    switch (num)
16    {
17        case 0:
18            HAL_GPIO_WritePin(
```

```
19         GPIOB,  
20         SEG_A_Pin | SEG_B_Pin | SEG_C_Pin |  
21         SEG_D_Pin | SEG_E_Pin | SEG_F_Pin,  
22         GPIO_PIN_RESET  
23     );  
24     break;  
25  
26     case 1:  
27         HAL_GPIO_WritePin(  
28             GPIOB,  
29             SEG_B_Pin | SEG_C_Pin,  
30             GPIO_PIN_RESET  
31         );  
32         break;  
33  
34     case 2:  
35         HAL_GPIO_WritePin(  
36             GPIOB,  
37             SEG_A_Pin | SEG_B_Pin | SEG_D_Pin |  
38             SEG_E_Pin | SEG_G_Pin,  
39             GPIO_PIN_RESET  
40         );  
41         break;  
42  
43     case 3:  
44         HAL_GPIO_WritePin(  
45             GPIOB,  
46             SEG_A_Pin | SEG_B_Pin | SEG_C_Pin |  
47             SEG_D_Pin | SEG_G_Pin,  
48             GPIO_PIN_RESET  
49         );  
50         break;  
51  
52     case 4:  
53         HAL_GPIO_WritePin(  
54             GPIOB,  
55             SEG_B_Pin | SEG_C_Pin |  
56             SEG_F_Pin | SEG_G_Pin,  
57             GPIO_PIN_RESET  
58         );  
59         break;  
60  
61     case 5:  
62         HAL_GPIO_WritePin(  
63             GPIOB,  
64             SEG_A_Pin | SEG_C_Pin | SEG_D_Pin |  
65             SEG_F_Pin | SEG_G_Pin,  
66             GPIO_PIN_RESET  
67         );  
68         break;  
69  
70     case 6:  
71         HAL_GPIO_WritePin(  
72             GPIOB,  
73             SEG_A_Pin | SEG_C_Pin | SEG_D_Pin |  
74             SEG_E_Pin | SEG_F_Pin | SEG_G_Pin,
```

```
75         GPIO_PIN_RESET
76     );
77     break;
78
79     case 7:
80         HAL_GPIO_WritePin(
81             GPIOB,
82             SEG_A_Pin | SEG_B_Pin | SEG_C_Pin,
83             GPIO_PIN_RESET
84         );
85         break;
86
87     case 8:
88         HAL_GPIO_WritePin(
89             GPIOB,
90             SEG_A_Pin | SEG_B_Pin | SEG_C_Pin |
91             SEG_D_Pin | SEG_E_Pin | SEG_F_Pin |
92             SEG_G_Pin,
93             GPIO_PIN_RESET
94         );
95         break;
96
97     case 9:
98         HAL_GPIO_WritePin(
99             GPIOB,
100            SEG_A_Pin | SEG_B_Pin | SEG_C_Pin |
101            SEG_D_Pin | SEG_F_Pin | SEG_G_Pin,
102            GPIO_PIN_RESET
103        );
104        break;
105
106    default:
107        /*
108         * An invalid input leaves
109         * every segment switched off.
110         */
111        break;
112    }
113 }
```

The first parameter of `HAL_GPIO_WritePin()` is `GPIOB`, which selects GPIO port B. The second parameter contains the segment pins that must be controlled. The third parameter determines the output state. Since the display is common-anode, `GPIO_PIN_RESET` turns a selected segment on, while `GPIO_PIN_SET` turns it off.

The segment patterns used to display the decimal numbers are summarized in Table 9.

Table 9: Segment patterns for the seven-segment display

Number	Activated segments
0	a, b, c, d, e, f
1	b, c
2	a, b, d, e, g
3	a, b, c, d, g
4	b, c, f, g
5	a, c, d, f, g
6	a, c, d, e, f, g
7	a, b, c
8	a, b, c, d, e, f, g
9	a, b, c, d, f, g

A counter is initialized to zero before the infinite loop. The current counter value is passed to `display7SEG()`, after which the counter is increased by one. When the counter reaches ten, it is reset to zero.

Program 1.5: Displaying the numbers from zero to nine

```

1  int counter = 0;
2
3  while (1)
4  {
5      if (counter >= 10)
6      {
7          counter = 0;
8      }
9
10     /* Display the current counter value. */
11     display7SEG(counter);
12
13     /* Prepare the next number. */
14     counter++;
15
16     /* Keep the number visible for one second. */
17     HAL_Delay(1000);
18 }

```

The instruction `HAL_Delay(1000)` keeps each number visible for 1000 milliseconds. After displaying 9, the counter returns to 0, creating the following continuous sequence:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 0 \rightarrow \dots$$

1.5 Exercise 5: Four-Way Traffic Light with Countdown

1.5.1 Objective and system integration

Exercise 5 combines the four-way traffic-light controller from Exercise 3 with the common-anode seven-segment display from Exercise 4. The traffic-light controller determines the colour shown in the North-South and East-West directions, while the seven-segment display shows the number of seconds remaining in the current phase.

The GPIO assignments from the two previous exercises are retained. PA5 through PA10 control the twelve physical traffic-light LEDs. Two LEDs in the same direction pair share each

output because North and South always show the same colour, and East and West always show the same colour. PB0 through PB6 control segments a through g of the seven-segment display.

Table 10: GPIO and Proteus component assignments for Exercise 5

GPIO pin	CubeMX label	Proteus component	Function
PA5	NS_RED	D1, D4	North-South red
PA6	NS_YELLOW	D2, D5	North-South yellow
PA7	NS_GREEN	D3, D6	North-South green
PA8	EW_RED	D7, D10	East-West red
PA9	EW_YELLOW	D8, D11	East-West yellow
PA10	EW_GREEN	D9, D12	East-West green
PB0–PB6	SEG_A–SEG_G	7SEG-COM-ANODE	Segments a–g

Both parts of the circuit use active-low logic. Therefore, `GPIO_PIN_RESET` activates an LED or a seven-segment segment, whereas `GPIO_PIN_SET` deactivates it. The submitted Proteus schematic uses direct simulated connections and contains no external resistor components.

Only one seven-segment display is used, so it shows one countdown for the currently active traffic-light phase. A three-second green phase displays $3 \rightarrow 2 \rightarrow 1$, and a two-second yellow phase displays $2 \rightarrow 1$.

Table 11: Traffic-light sequence and countdown values for Exercise 5

State	North-South	East-West	Countdown	Duration
1	Green	Red	$3 \rightarrow 2 \rightarrow 1$	3 seconds
2	Yellow	Red	$2 \rightarrow 1$	2 seconds
3	Red	Green	$3 \rightarrow 2 \rightarrow 1$	3 seconds
4	Red	Yellow	$2 \rightarrow 1$	2 seconds

The North-South and East-West groups never receive green signals simultaneously. While one group is green or yellow, the perpendicular group remains red. One complete traffic-light cycle lasts ten seconds.

1.5.2 Schematic

Figure 5 presents the combined traffic-light and countdown circuit. It includes the twelve traffic-light LEDs and the single common-anode seven-segment display.

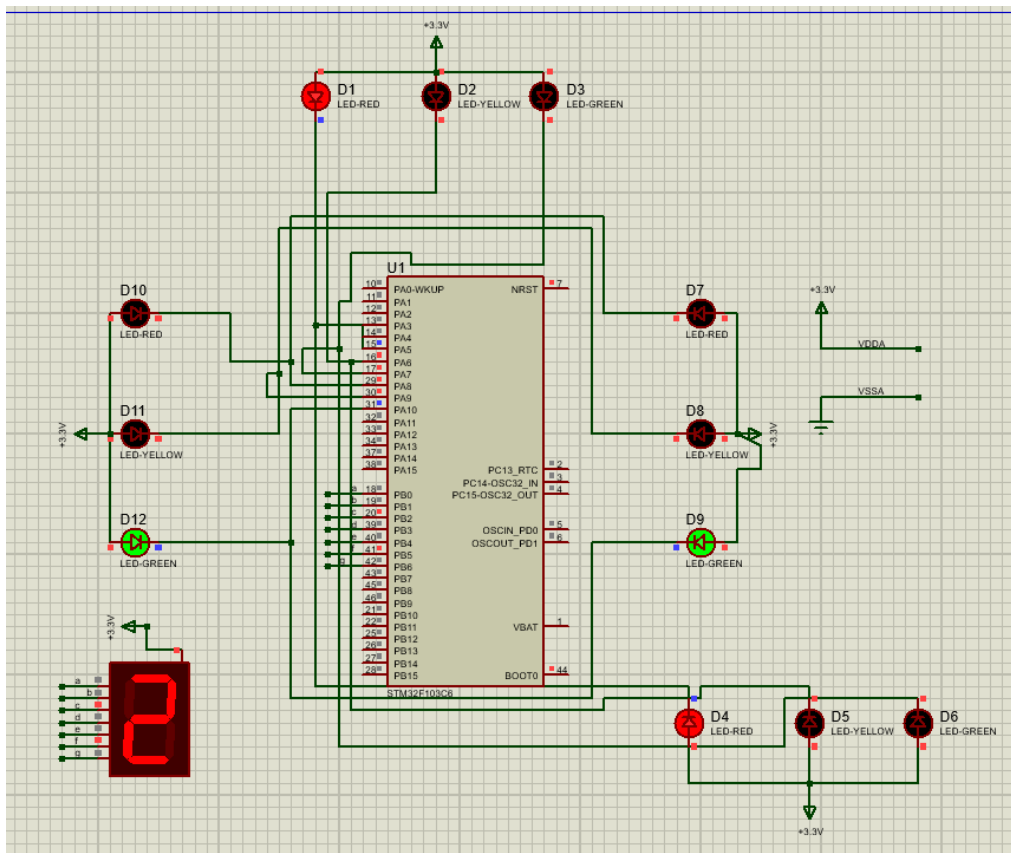


Figure 5: Exercise 5 Proteus schematic. Project files are provided in Reference [1].

1.5.3 Implementation and operation

The `display7SEG()` function developed in Exercise 4 is reused without modification. A new function named `countdown()` controls the displayed number and the duration of each traffic-light state.

The parameter `seconds` is the initial number of seconds for the phase. It is a local copy of the argument supplied by the caller. While its value is greater than zero, the function sends the value to `display7SEG()`, keeps it visible for 1000 ms, and decreases it by one. The loop stops when `seconds` becomes zero.

Program 1.6: Seven-segment countdown function

```
1 void countdown(int seconds)
2 {
3     while (seconds > 0)
4     {
5         /* Display the remaining time. */
6         display7SEG(seconds);
7
8         /* Keep the value visible for one second. */
9         HAL_Delay(1000);
10
11        /* Reduce the remaining time. */
12        seconds--;
13    }
```

14 }

Calling `countdown(3)` displays 3, 2, and 1, with each number visible for one second. The function therefore produces a total delay of three seconds. Similarly, `countdown(2)` produces a two-second delay while displaying 2 and 1.

Before each call to `countdown()`, the program writes all six logical traffic-light outputs explicitly. This establishes the required colour in both direction groups before the countdown begins and prevents an output from accidentally retaining a value from the previous state.

Program 1.7: Four-way traffic-light controller with countdown

```
1  while (1)
2  {
3      /*
4       * State 1:
5       * North-South GREEN, East-West RED
6       */
7      HAL_GPIO_WritePin(GPIOA, NS_RED_Pin,
8                          GPIO_PIN_SET);
9      HAL_GPIO_WritePin(GPIOA, NS_YELLOW_Pin,
10                         GPIO_PIN_SET);
11     HAL_GPIO_WritePin(GPIOA, NS_GREEN_Pin,
12                        GPIO_PIN_RESET);
13
14     HAL_GPIO_WritePin(GPIOA, EW_RED_Pin,
15                        GPIO_PIN_RESET);
16     HAL_GPIO_WritePin(GPIOA, EW_YELLOW_Pin,
17                        GPIO_PIN_SET);
18     HAL_GPIO_WritePin(GPIOA, EW_GREEN_Pin,
19                        GPIO_PIN_SET);
20
21     countdown(3);
22
23     /*
24      * State 2:
25      * North-South YELLOW, East-West RED
26      */
27     HAL_GPIO_WritePin(GPIOA, NS_RED_Pin,
28                        GPIO_PIN_SET);
29     HAL_GPIO_WritePin(GPIOA, NS_YELLOW_Pin,
30                        GPIO_PIN_RESET);
31     HAL_GPIO_WritePin(GPIOA, NS_GREEN_Pin,
32                        GPIO_PIN_SET);
33
34     HAL_GPIO_WritePin(GPIOA, EW_RED_Pin,
35                        GPIO_PIN_RESET);
36     HAL_GPIO_WritePin(GPIOA, EW_YELLOW_Pin,
37                        GPIO_PIN_SET);
38     HAL_GPIO_WritePin(GPIOA, EW_GREEN_Pin,
39                        GPIO_PIN_SET);
40
41     countdown(2);
42
43     /*
```

```
44      * State 3:
45      * North-South RED, East-West GREEN
46      */
47      HAL_GPIO_WritePin(GPIOA, NS_RED_Pin,
48                          GPIO_PIN_RESET);
49      HAL_GPIO_WritePin(GPIOA, NS_YELLOW_Pin,
50                          GPIO_PIN_SET);
51      HAL_GPIO_WritePin(GPIOA, NS_GREEN_Pin,
52                          GPIO_PIN_SET);
53
54      HAL_GPIO_WritePin(GPIOA, EW_RED_Pin,
55                          GPIO_PIN_SET);
56      HAL_GPIO_WritePin(GPIOA, EW_YELLOW_Pin,
57                          GPIO_PIN_SET);
58      HAL_GPIO_WritePin(GPIOA, EW_GREEN_Pin,
59                          GPIO_PIN_RESET);
60
61      countdown(3);
62
63      /*
64      * State 4:
65      * North-South RED, East-West YELLOW
66      */
67      HAL_GPIO_WritePin(GPIOA, NS_RED_Pin,
68                          GPIO_PIN_RESET);
69      HAL_GPIO_WritePin(GPIOA, NS_YELLOW_Pin,
70                          GPIO_PIN_SET);
71      HAL_GPIO_WritePin(GPIOA, NS_GREEN_Pin,
72                          GPIO_PIN_SET);
73
74      HAL_GPIO_WritePin(GPIOA, EW_RED_Pin,
75                          GPIO_PIN_SET);
76      HAL_GPIO_WritePin(GPIOA, EW_YELLOW_Pin,
77                          GPIO_PIN_RESET);
78      HAL_GPIO_WritePin(GPIOA, EW_GREEN_Pin,
79                          GPIO_PIN_SET);
80
81      countdown(2);
82  }
```

After the fourth state finishes, the infinite loop returns to the first state. The complete sequence repeats continuously while the seven-segment display provides a visible countdown for every traffic-light phase. Each direction is red for five seconds while the perpendicular direction completes its three-second green phase and two-second yellow phase. No all-red interval is inserted between the four states in this implementation.

1.6 Exercise 6: Analog Clock LED Connection Test

1.6.1 Objective and GPIO configuration

Exercise 6 uses twelve LEDs arranged in the shape of an analog clock. The LEDs represent the twelve positions on a clock face and are connected sequentially from PA4 to PA15 of the STM32F103C6Ux microcontroller.

The purpose of the program is to test every LED connection. Only one LED is activated at a time. After a delay of 500 milliseconds, the active LED is deactivated and the next LED is

activated.

The LEDs use the same active-low connection as the previous exercises. A low GPIO output activates an LED, while a high GPIO output deactivates it.

Table 12: GPIO configuration for Exercise 6

Clock position	Proteus component	GPIO pin	CubeMX label
1	D1	PA4	CLOCK_LED_1
2	D2	PA5	CLOCK_LED_2
3	D3	PA6	CLOCK_LED_3
4	D4	PA7	CLOCK_LED_4
5	D5	PA8	CLOCK_LED_5
6	D6	PA9	CLOCK_LED_6
7	D7	PA10	CLOCK_LED_7
8	D8	PA11	CLOCK_LED_8
9	D9	PA12	CLOCK_LED_9
10	D10	PA13	CLOCK_LED_10
11	D11	PA14	CLOCK_LED_11
12	D12	PA15	CLOCK_LED_12

PA13 and PA14 are normally reserved for the Serial Wire Debug interface, while PA15 can be reserved for JTAG. Therefore, the debug mode in STM32CubeMX is set to **No Debug**, allowing PA13, PA14, and PA15 to be configured as normal GPIO outputs.

The submitted Proteus schematic connects the LEDs directly and does not use external resistor components. The initial GPIO output level is set to high so that every active-low LED remains off when the system starts.

1.6.2 Schematic

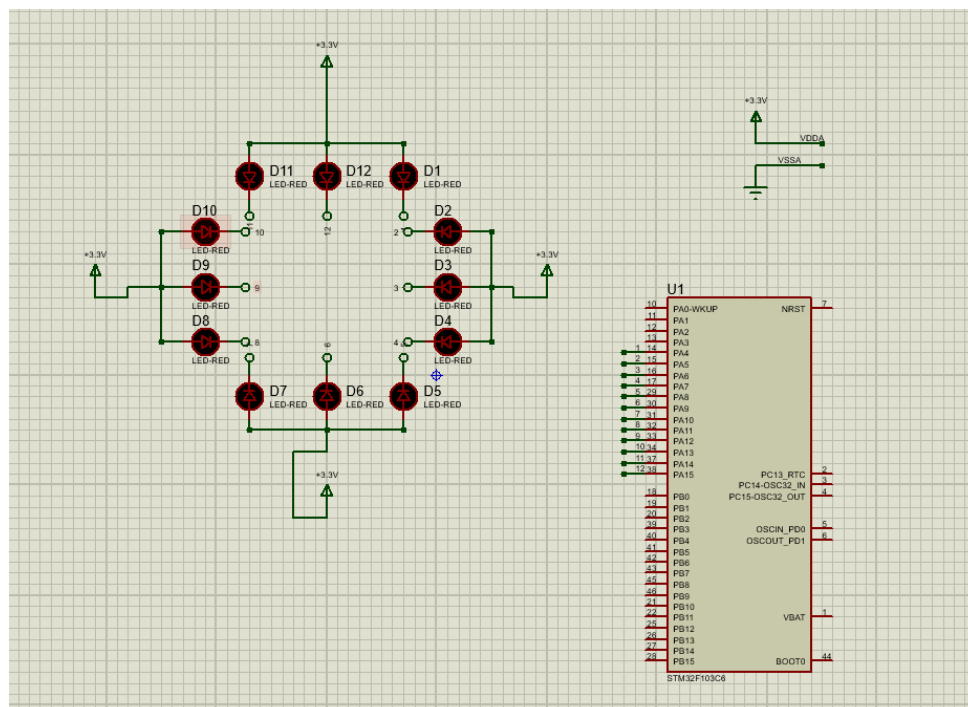


Figure 6: Exercise 6 - 10 Proteus schematic. Project files are provided in Reference [1].

1.6.3 Implementation and operation

A reusable function named `testClockLED()` is used to test one LED. The function receives the GPIO pin of the selected LED and the time for which the LED should remain on.

Program 1.8: Function for testing one clock LED

```
1 void testClockLED(uint16_t pin, uint32_t delay_ms)
2 {
3     /* Active-low: logic 0 turns the LED on. */
4     HAL_GPIO_WritePin(GPIOA, pin,
5                       GPIO_PIN_RESET);
6
7     /* Keep the selected LED on. */
8     HAL_Delay(delay_ms);
9
10    /* Active-low: logic 1 turns the LED off. */
11    HAL_GPIO_WritePin(GPIOA, pin,
12                      GPIO_PIN_SET);
13 }
```

The function declaration contains two parameters:

- `uint16_t pin` contains the GPIO pin mask of the LED that should be tested. For example, `CLOCK_LED_1_Pin` represents the LED connected to PA4.
- `uint32_t delay_ms` specifies how long the selected LED remains on. The value is measured in milliseconds.

All clock LEDs are connected to GPIO port A. Therefore, the port is fixed as `GPIOA` inside the function and does not need to be passed as another parameter.

The first call to `HAL_GPIO_WritePin()` produces a low output and activates the selected LED. The function then waits for the number of milliseconds specified by `delay_ms`. Finally, the second call produces a high output and deactivates the LED.

Before the test sequence begins, all twelve LEDs are explicitly deactivated.

Program 1.9: Deactivating all clock LEDs

```
1 HAL_GPIO_WritePin(
2     GPIOA,
3     CLOCK_LED_1_Pin | CLOCK_LED_2_Pin |
4     CLOCK_LED_3_Pin | CLOCK_LED_4_Pin |
5     CLOCK_LED_5_Pin | CLOCK_LED_6_Pin |
6     CLOCK_LED_7_Pin | CLOCK_LED_8_Pin |
7     CLOCK_LED_9_Pin | CLOCK_LED_10_Pin |
8     CLOCK_LED_11_Pin | CLOCK_LED_12_Pin,
9     GPIO_PIN_SET
10 );
```

The bitwise OR operator (`|`) combines the twelve GPIO pin masks. Consequently, all twelve output pins can be set high using one function call.

The infinite loop calls `testClockLED()` once for each LED. Each call uses a delay value of 500 milliseconds.

Program 1.10: Sequential test of the twelve clock LEDs

```
1 while (1)
2 {
3     testClockLED(CLOCK_LED_1_Pin, 500);
4     testClockLED(CLOCK_LED_2_Pin, 500);
5     testClockLED(CLOCK_LED_3_Pin, 500);
6     testClockLED(CLOCK_LED_4_Pin, 500);
7     testClockLED(CLOCK_LED_5_Pin, 500);
8     testClockLED(CLOCK_LED_6_Pin, 500);
9     testClockLED(CLOCK_LED_7_Pin, 500);
10    testClockLED(CLOCK_LED_8_Pin, 500);
11    testClockLED(CLOCK_LED_9_Pin, 500);
12    testClockLED(CLOCK_LED_10_Pin, 500);
13    testClockLED(CLOCK_LED_11_Pin, 500);
14    testClockLED(CLOCK_LED_12_Pin, 500);
15 }
```

The resulting sequence begins at clock LED 1 and continues through clock LED 12. After the twelfth LED is tested, the infinite loop returns to the first LED and repeats the sequence continuously:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow 11 \rightarrow 12 \rightarrow 1 \rightarrow \dots$$

Since each LED remains on for 500 milliseconds, one complete twelve-LED test cycle lasts six seconds.

1.7 Exercise 7: Clearing All Clock LEDs

1.7.1 Objective

The objective of Exercise 7 is to implement a function named `clearAllClock()` that simultaneously deactivates all twelve LEDs of the analog clock developed in Exercise 6.

The GPIO configuration remains unchanged. The twelve clock LEDs are connected to PA4 through PA15 and operate using active-low logic. Therefore, `GPIO_PIN_SET` deactivates an LED, while `GPIO_PIN_RESET` activates it.

1.7.2 Implementation and operation

The `clearAllClock()` function does not require any input parameters because it always performs the same operation on all twelve clock LEDs. It also does not return a value.

Program 1.11: Function for turning off all clock LEDs

```
1 void clearAllClock(void)
2 {
3     HAL_GPIO_WritePin(
4         GPIOA,
5         CLOCK_LED_1_Pin | CLOCK_LED_2_Pin |
6         CLOCK_LED_3_Pin | CLOCK_LED_4_Pin |
7         CLOCK_LED_5_Pin | CLOCK_LED_6_Pin |
8         CLOCK_LED_7_Pin | CLOCK_LED_8_Pin |
9         CLOCK_LED_9_Pin | CLOCK_LED_10_Pin |
10        CLOCK_LED_11_Pin | CLOCK_LED_12_Pin,
```

```
11         GPIO_PIN_SET  
12     );  
13 }
```

In the function declaration, the first `void` indicates that the function does not return a value. The `void` inside the parentheses indicates that the function does not receive any input parameters.

The call to `HAL_GPIO_WritePin()` contains three parameters:

- `GPIOA` selects GPIO port A, which contains all twelve clock LED outputs.
- The second parameter combines the pin masks for PA4 through PA15. The bitwise OR operator (`|`) allows all twelve pins to be selected in one function call.
- `GPIO_PIN_SET` writes logic 1 to all selected pins. Since the LEDs are active-low, this deactivates every clock LED.

The function can be called using the following instruction:

Program 1.12: Calling the clear function

```
1 clearAllClock();
```

When this instruction is executed, all twelve clock LEDs are turned off simultaneously. GPIO pins PA0 through PA3 are not included in the pin mask and are therefore not affected by the function.

1.8 Exercise 8: Activating a Selected Clock LED

1.8.1 Objective

The objective of Exercise 8 is to implement a function named `setNumberOnClock(int num)`. The function receives an integer from 0 to 11 and activates the corresponding LED of the twelve-LED analog clock.

The input follows clock-position numbering. Position 0 represents the 12-o'clock position, while positions 1 through 11 represent the corresponding numbers on the clock face.

The clock LEDs operate using active-low logic. Consequently, the selected LED is activated by writing `GPIO_PIN_RESET` to its GPIO output.

Table 13: Input-to-LED mapping for Exercise 8

Input	Proteus component	GPIO pin	CubeMX label
0	D12	PA15	CLOCK_LED_12
1	D1	PA4	CLOCK_LED_1
2	D2	PA5	CLOCK_LED_2
3	D3	PA6	CLOCK_LED_3
4	D4	PA7	CLOCK_LED_4
5	D5	PA8	CLOCK_LED_5
6	D6	PA9	CLOCK_LED_6
7	D7	PA10	CLOCK_LED_7
8	D8	PA11	CLOCK_LED_8
9	D9	PA12	CLOCK_LED_9
10	D10	PA13	CLOCK_LED_10
11	D11	PA14	CLOCK_LED_11

1.8.2 Implementation and operation

The function receives one parameter named `num`. Its valid input range is from 0 to 11. A `switch` statement selects the GPIO pin associated with the requested clock position.

Program 1.13: Function for activating a selected clock LED

```

1 void setNumberOnClock(int num)
2 {
3     switch (num)
4     {
5         case 0:
6             /* Position 0 represents 12 o'clock. */
7             HAL_GPIO_WritePin(
8                 GPIOA,
9                 CLOCK_LED_12_Pin,
10                GPIO_PIN_RESET
11            );
12            break;
13
14         case 1:
15             HAL_GPIO_WritePin(
16                 GPIOA,
17                 CLOCK_LED_1_Pin,
18                 GPIO_PIN_RESET
19            );
20            break;
21
22         case 2:
23             HAL_GPIO_WritePin(
24                 GPIOA,
25                 CLOCK_LED_2_Pin,
26                 GPIO_PIN_RESET
27            );
28            break;
29
30         case 3:
31             HAL_GPIO_WritePin(
32                 GPIOA,

```

```
33         CLOCK_LED_3_Pin,  
34         GPIO_PIN_RESET  
35     );  
36     break;  
37  
38     case 4:  
39         HAL_GPIO_WritePin(  
40             GPIOA,  
41             CLOCK_LED_4_Pin,  
42             GPIO_PIN_RESET  
43         );  
44         break;  
45  
46     case 5:  
47         HAL_GPIO_WritePin(  
48             GPIOA,  
49             CLOCK_LED_5_Pin,  
50             GPIO_PIN_RESET  
51         );  
52         break;  
53  
54     case 6:  
55         HAL_GPIO_WritePin(  
56             GPIOA,  
57             CLOCK_LED_6_Pin,  
58             GPIO_PIN_RESET  
59         );  
60         break;  
61  
62     case 7:  
63         HAL_GPIO_WritePin(  
64             GPIOA,  
65             CLOCK_LED_7_Pin,  
66             GPIO_PIN_RESET  
67         );  
68         break;  
69  
70     case 8:  
71         HAL_GPIO_WritePin(  
72             GPIOA,  
73             CLOCK_LED_8_Pin,  
74             GPIO_PIN_RESET  
75         );  
76         break;  
77  
78     case 9:  
79         HAL_GPIO_WritePin(  
80             GPIOA,  
81             CLOCK_LED_9_Pin,  
82             GPIO_PIN_RESET  
83         );  
84         break;  
85  
86     case 10:  
87         HAL_GPIO_WritePin(  
88             GPIOA,
```

```
89         CLOCK_LED_10_Pin,  
90         GPIO_PIN_RESET  
91     );  
92     break;  
93  
94     case 11:  
95         HAL_GPIO_WritePin(  
96             GPIOA,  
97             CLOCK_LED_11_Pin,  
98             GPIO_PIN_RESET  
99         );  
100        break;  
101  
102        default:  
103            /* Invalid input: do nothing. */  
104            break;  
105    }  
106 }
```

The return type `void` indicates that the function does not return a value. The parameter `int num` contains the clock position that should be activated.

Input 0 is treated specially because it represents the 12-o'clock position. Therefore, input 0 selects `CLOCK_LED_12_Pin`, which is connected to PA15. Inputs 1 through 11 directly select the corresponding LEDs.

Every call to `HAL_GPIO_WritePin()` contains three parameters:

- `GPIOA` selects GPIO port A.
- The second parameter selects the GPIO pin connected to the requested clock position.
- `GPIO_PIN_RESET` outputs logic 0 and activates the selected active-low LED.

The function can be called using instructions such as:

Program 1.14: Calling the LED activation function

```
1  setNumberOnClock(0);  /* Turn on D12 */  
2  setNumberOnClock(5);  /* Turn on D5  */  
3  setNumberOnClock(11); /* Turn on D11 */
```

The function does not automatically deactivate the other clock LEDs. This allows separate LEDs to remain active for the hour, minute, and second indicators.

1.9 Exercise 9: Deactivating a Selected Clock LED

1.9.1 Objective

The objective of Exercise 9 is to implement a function named `clearNumberOnClock(int num)`. The function receives an integer from 0 to 11 and deactivates the corresponding LED of the twelve-LED analog clock.

The function uses the same clock-position mapping as `setNumberOnClock()` from Exercise 8. Position 0 represents the 12-o'clock position. Positions 1 through 11 represent the corresponding numbers on the clock face.

The LEDs operate using active-low logic. Therefore, the selected LED is deactivated by writing `GPIO_PIN_SET` to its GPIO output.

Table 14: Input-to-LED mapping for Exercise 9

Input	Proteus component	GPIO pin	CubeMX label
0	D12	PA15	CLOCK_LED_12
1	D1	PA4	CLOCK_LED_1
2	D2	PA5	CLOCK_LED_2
3	D3	PA6	CLOCK_LED_3
4	D4	PA7	CLOCK_LED_4
5	D5	PA8	CLOCK_LED_5
6	D6	PA9	CLOCK_LED_6
7	D7	PA10	CLOCK_LED_7
8	D8	PA11	CLOCK_LED_8
9	D9	PA12	CLOCK_LED_9
10	D10	PA13	CLOCK_LED_10
11	D11	PA14	CLOCK_LED_11

1.9.2 Implementation and operation

The function receives one parameter named `num`. A `switch` statement selects the LED associated with the requested clock position. Only the selected LED is deactivated; the other LEDs are not modified.

Program 1.15: Function for deactivating a selected clock LED

```

1 void clearNumberOnClock(int num)
2 {
3     switch (num)
4     {
5         case 0:
6             /* Position 0 represents 12 o'clock. */
7             HAL_GPIO_WritePin(
8                 GPIOA,
9                 CLOCK_LED_12_Pin,
10                GPIO_PIN_SET
11            );
12            break;
13
14         case 1:
15             HAL_GPIO_WritePin(
16                 GPIOA,
17                 CLOCK_LED_1_Pin,
18                 GPIO_PIN_SET
19            );
20            break;
21
22         case 2:
23             HAL_GPIO_WritePin(
24                 GPIOA,
25                 CLOCK_LED_2_Pin,
26                 GPIO_PIN_SET
27            );
28            break;

```

```
29
30     case 3:
31         HAL_GPIO_WritePin(
32             GPIOA,
33             CLOCK_LED_3_Pin,
34             GPIO_PIN_SET
35         );
36         break;
37
38     case 4:
39         HAL_GPIO_WritePin(
40             GPIOA,
41             CLOCK_LED_4_Pin,
42             GPIO_PIN_SET
43         );
44         break;
45
46     case 5:
47         HAL_GPIO_WritePin(
48             GPIOA,
49             CLOCK_LED_5_Pin,
50             GPIO_PIN_SET
51         );
52         break;
53
54     case 6:
55         HAL_GPIO_WritePin(
56             GPIOA,
57             CLOCK_LED_6_Pin,
58             GPIO_PIN_SET
59         );
60         break;
61
62     case 7:
63         HAL_GPIO_WritePin(
64             GPIOA,
65             CLOCK_LED_7_Pin,
66             GPIO_PIN_SET
67         );
68         break;
69
70     case 8:
71         HAL_GPIO_WritePin(
72             GPIOA,
73             CLOCK_LED_8_Pin,
74             GPIO_PIN_SET
75         );
76         break;
77
78     case 9:
79         HAL_GPIO_WritePin(
80             GPIOA,
81             CLOCK_LED_9_Pin,
82             GPIO_PIN_SET
83         );
84         break;
```

```
85
86     case 10:
87         HAL_GPIO_WritePin(
88             GPIOA,
89             CLOCK_LED_10_Pin,
90             GPIO_PIN_SET
91         );
92         break;
93
94     case 11:
95         HAL_GPIO_WritePin(
96             GPIOA,
97             CLOCK_LED_11_Pin,
98             GPIO_PIN_SET
99         );
100        break;
101
102    default:
103        /* Invalid input: do nothing. */
104        break;
105 }
106 }
```

The return type `void` indicates that the function does not return a value. The parameter `int num` contains the clock position that should be deactivated.

Input 0 selects `CLOCK_LED_12_Pin`, which represents the 12-o'clock position. Inputs 1 through 11 select the corresponding numbered LEDs.

Every call to `HAL_GPIO_WritePin()` contains three parameters:

- `GPIOA` selects GPIO port A.
- The second parameter selects the GPIO pin connected to the requested clock position.
- `GPIO_PIN_SET` outputs logic 1 and deactivates the selected active-low LED.

The function can be called using instructions such as:

Program 1.16: Calling the LED deactivation function

```
1 clearNumberOnClock(0); /* Turn off D12 */
2 clearNumberOnClock(5); /* Turn off D5 */
3 clearNumberOnClock(11); /* Turn off D11 */
```

The function only deactivates the selected clock position. It does not modify the other eleven LEDs, allowing the hour, minute, and second indicators to be updated independently.

1.10 Exercise 10: Integrated Analog Clock

1.10.1 Objective and control strategy

Exercise 10 integrates the twelve-LED circuit and the functions developed in Exercises 6 through 9 to create a simulated analog clock. The twelve LEDs represent twelve clock positions indexed from 0 to 11.

The clock-position indexing follows the physical clock face. Position 0 represents the 12-o'clock position and is displayed by `CLOCK_LED_12`, connected to PA15. Positions 1 through 11 are displayed by `CLOCK_LED_1` through `CLOCK_LED_11`, respectively.

The program maintains three variables for the current hour, minute, and second. These time values are converted into clock positions before the corresponding LEDs are activated.

The previously implemented functions are reused:

- `clearAllClock()` deactivates all twelve clock LEDs.
- `setNumberOnClock(int num)` activates one selected clock position.
- `clearNumberOnClock(int num)` deactivates one selected clock position.

The hour value is converted using the remainder operator, while the minute and second values are converted using integer division by five.

Table 15: Conversion of time values into clock positions

Information	Calculation	Result range
Hour	<code>hour % 12</code>	0–11
Minute	<code>minute / 5</code>	0–11
Second	<code>second / 5</code>	0–11

Because one complete clock revolution contains twelve LED positions, every five-minute or five-second interval corresponds to one position:

$$\frac{60}{12} = 5.$$

The clock begins at 10:25:40 in this implementation. This starting value can be changed by modifying the initial values of the time variables.

1.10.2 Implementation and operation

The time and position variables are declared inside the `main()` function.

Program 1.17: Time and clock-position variables

```
1  /* Initial simulation time: 10:25:40 */
2  int hour = 10;
3  int minute = 25;
4  int second = 40;
5
6  int hour_position;
7  int minute_position;
8  int second_position;
```

The variables `hour`, `minute`, and `second` contain the current simulated time. The other three variables contain the corresponding clock positions from 0 to 11.

The resulting position mapping is:

- Position 0 activates D12 at the 12-o'clock position.
- Position 1 activates D1 at the 1-o'clock position.
- Position 2 activates D2 at the 2-o'clock position.

- The same relationship continues until position 11 activates D11.
After GPIO initialization, all clock LEDs are deactivated to guarantee a known initial state.

Program 1.18: Initializing the clock display

```
1  /* Execute this after MX_GPIO_Init(). */
2  clearAllClock();
```

The main clock loop converts the current time into clock positions, activates the three logical indicators, waits for one second, and then removes the previously displayed positions.

Program 1.19: Integrated analog-clock program

```
1  while (1)
2  {
3      /*
4          * Convert the current time into
5          * clock positions from 0 to 11.
6          *
7          * Position 0 represents 12 o'clock.
8          */
9      hour_position = hour % 12;
10     minute_position = minute / 5;
11     second_position = second / 5;
12
13     /*
14         * Display the hour, minute and second positions.
15         */
16     setNumberOnClock(hour_position);
17     setNumberOnClock(minute_position);
18     setNumberOnClock(second_position);
19
20     /* Keep the current time displayed for one second. */
21     HAL_Delay(1000);
22
23     /*
24         * Remove the previously displayed positions.
25         */
26     clearNumberOnClock(hour_position);
27     clearNumberOnClock(minute_position);
28     clearNumberOnClock(second_position);
29
30     /* Advance the clock by one second. */
31     second++;
32
33     /* Change 60 seconds into one minute. */
34     if (second >= 60)
35     {
36         second = 0;
37         minute++;
38
39         /* Change 60 minutes into one hour. */
40         if (minute >= 60)
41         {
```

```
42         minute = 0;  
43         hour++;  
44  
45         /* Restart after 23:59:59. */  
46         if (hour >= 24)  
47         {  
48             hour = 0;  
49         }  
50     }  
51 }  
52 }
```

For example, at 10:25:40, the clock positions are calculated as follows:

$\text{hour position} = 10 \bmod 12 = 10,$
 $\text{minute position} = 25/5 = 5,$
 $\text{second position} = 40/5 = 8.$

Using the corrected clock-position mapping, the program performs the following operations:

- `setNumberOnClock(10)` activates D10 for the hour.
- `setNumberOnClock(5)` activates D5 for the minute.
- `setNumberOnClock(8)` activates D8 for the second.

Therefore, the initial time 10:25:40 is represented by D10, D5, and D8. After one second, these positions are deactivated, the second value is incremented, and the updated positions are displayed.

The time rollover conditions maintain valid clock values. When the second value reaches 60, it is reset to 0 and the minute value is increased. When the minute value reaches 60, it is reset to 0 and the hour value is increased. Finally, when the hour value reaches 24, it returns to 0.

At most three physical LEDs are active at one time. If the hour, minute, or second indicators select the same clock position, fewer than three physical LEDs are visible because the logical indicators overlap on the same LED.

References

- [1] V. L. T. Trieu, *STM32 Lab 01: Exercises 1–10*, GitHub repository. [Online]. Available: <https://github.com/volethanhtrieu/Lab-exercise-MPU---MCU/tree/main/lab01>. [Accessed: September 18, 2026].
- [2] L. T. Nhan, *Microcontroller Laboratory – Lab 01: LED Animations*, Ho Chi Minh City University of Technology, 2021.
- [3] STMicroelectronics, *STM32F103x4 and STM32F103x6 Low-density Performance Line Datasheet*, DocID15060, Rev. 7, June 2015. [Online]. Available: <https://www.st.com/resource/en/datasheet/stm32f103c4.pdf>. [Accessed: September 18, 2026].
- [4] STMicroelectronics, *STM32CubeMX: STM32Cube Initialization Code Generator*. [Online]. Available: <https://www.st.com/en/development-tools/stm32cubemx.html>. [Accessed: September 18, 2026].
- [5] STMicroelectronics, *STM32CubeF1 Embedded Software Package*. [Online]. Available: <https://www.st.com/en/embedded-software/stm32cubef1.html>. [Accessed: September 18, 2026].
- [6] Labcenter Electronics, *Proteus Circuit Simulation Software with SPICE*. [Online]. Available: <https://www.labcenter.com/simulation/>. [Accessed: September 18, 2026].